

delete_wifi_profile

REFERENCE



API consumer



permanently remove a saved Wi-Fi profile from the reader by ESSID

Overview

`delete_wifi_profile` permanently removes a single saved Wi-Fi profile from the reader's profile store. The profile is identified by its `essid` inside `wifiProfileInfo`; on success the matching saved profile — including any stored credentials — is deleted, and the command returns a status-only envelope (no profile echo). It is a state-changing command on the device-management ([MGMT](#)) endpoint. Confirm the result with `get_wifi`.

Deletion is permanent — there is no "undelete"; restoring a profile means re-provisioning with `set_wifi`. The deletion is safe for the control session: the reader refuses to delete the active profile ([code 16](#)). See [Notes and Caveats](#).

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	STATE-CHANGING (removes a saved profile and its stored credentials)
Reversibility	IRREVERSIBLE (there is no undo; re-provision with <code>set_wifi</code> to restore a deleted profile)
Required Request Fields	<code>command</code> , <code>requestId</code> , <code>wifiProfileInfo</code> (with <code>essid</code> to identify the profile)
Applies To	Endpoint: Management (MGMT). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	<code>get_wifi</code> , <code>set_wifi</code> , <code>delete_certificate</code>

When to Use

Use `delete_wifi_profile` to **clean up the reader's saved Wi-Fi profiles** — one profile per call, matched by `ssid`.

- **Remove an obsolete or incorrect profile** — e.g. a network that no longer exists, or one saved with the wrong credentials.
- **Make room before adding a profile** — the device enforces a maximum number of saved profiles; if `set_wifi` `CREATE` returns [code 19](#) (SSID count overflow), delete an unused profile first.
- **Tidy fallback sets** — prune stale entries so the reader only carries the profiles you intend it to roam across.

Delete only a **non-active** profile. To retire the network the reader is currently connected to, first connect it elsewhere (with `set_wifi` `connect: true`) — the device will not delete the profile carrying its live connection ([code 16](#)).

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/MGMT/clients/cmnd/<serial></code>	On the management endpoint.
Response	Reader → App	<code><tenantId>/MGMT/clients/resp/<serial></code>	Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Credentials are presented once at CONNECT; `delete_wifi_profile` therefore carries **no credential field**, so there is nothing to mask. The `requestId` you send is echoed in the response so you can pair reply to request. The endpoint's `mqttParams` govern QoS and retain, not this command.

See [Topics and Channels](#).

Key Fields

- `wifiProfileInfo.essid` (request) — the network name of the saved profile to delete. Case-sensitive; must exactly match a saved profile (verify the exact name with `get_wifi`). Must name a non-active profile; targeting the active profile fails with [code 16](#).
- `response.code` (response) — read this to determine success vs failure: `0` (deleted), `15` (no such saved profile), `16` (that profile is the active connection), `17` (no `essid` supplied). See [Response Codes](#).

Request Payload

The request envelope is `command`, `requestId`, and the `wifiProfileInfo` object naming the profile to delete. Use the **Schema** tab for the full field reference and the **Example** tab for a ready-to-send request.

Request payload

```
1 {
2   "command" : "delete_wifi_profile",
3   "requestId" : "abc123",
4   "wifiProfileInfo" : {
5     "essid" : "TestAP2"
6   }
7 }
```

Request payload

* required

— not applicable

Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command and	string	—	<code>delete_wifi_profile</code>	Fixed command name.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
2	requestId *	string	—	abc123	A unique identifier for the request; echoed back in the response so you can pair reply to request.
3	wifiProfileInfo *	object	—	—	Identifies the saved Wi-Fi profile to delete. Required by the contract, and it must name a profile with <code>essid</code> (see Request Semantics).
4	essid *	string	—	JioFi	The network name (ESSID) of the saved profile to remove. Case-sensitive; must exactly match a saved profile.

Request Semantics

- **One profile per call, matched by `essid`.** `delete_wifi_profile` removes a single saved profile identified by its network name. A `code 15` means the `essid` is not saved (typo or wrong case); confirm the exact stored name with `get_wifi` first. A `code 17` means the request reached the device with no `essid` at all.
- `wifiProfileInfo` **is required**. The contract requires it, so a request omitting it is invalid — and it would name no profile and delete nothing in any case. Always send `wifiProfileInfo` with a valid `essid` (which **is required** within `wifiProfileInfo`).

Response Payload

`delete_wifi_profile` returns a **status-only** envelope — `command`, `requestId`, `response` and, optionally, `apiVersion`. The contract's schema for this reply does not declare `apiVersion` at all; the published success example omits it and the error examples carry it. There is no profile echo; to confirm the deletion, follow up with `get_wifi` and check that the `essid` is gone. Pick a **result** below; the Example and Schema update together — the variants differ only in `response.code` / `response.description` and in whether `apiVersion` is present. Parse `apiVersion` as optional; see the [response envelope](#).

Result

Successful deletion

▼

Response payload

```
1 {
2   "command" : "delete_wifi_profile" ,
3   "requestId" : "abc345" ,
4   "response" : {
5     "code" : 0 ,
6     "description" : "Success"
7   }
8 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	string	—	delete_wifi_profile	Echoes the executed command.
2	requestId *	string	—	abc345	Echoes the request's requestId — pairs reply to request.
3	response *	object	—	—	Status-only result — delete_wifi_profile echoes no profile data.

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
4	code*	integer	0-101	0	0 = Success; non-zero = error. Required by the contract. See Response Codes below.
5	description*	string	—	Success	Human-readable status.

Response Codes

`delete_wifi_profile` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the matching profile was removed.
2	Invalid payload (<code>IOT_ERROR_INVALID_PAYLOAD</code>) — the JSON is malformed or has invalid field values. Validate against the schema and correct.
15	WIFI Error - SSID not found (<code>IOT_ERROR_SSID_NOT_FOUND</code>) — the <code>ssid</code> is not a saved profile. Verify the exact name (case-sensitive) with <code>get_wifi</code> .
16	WIFI Error - Cannot delete active SSID (<code>IOT_ERROR_DELETE_ACTIVE_SSID</code>) — the target <code>ssid</code> is the currently-connected profile; the reader protects it. Connect elsewhere first, then delete.

Code	Description
17	WIFI Error - SSID missed (<code>IOT_ERROR_SSID_MISSED</code>) — no <code>ssid</code> was supplied in <code>wifiProfileInfo</code> . Include the <code>ssid</code> field.
101	Default error — generic catch-all.

Examples

Publish a `delete_wifi_profile` request naming the `ssid` to remove; the device replies with the status-only envelope. The **Result** dropdown under [Response Payload](#) shows the three replies in the contract:

- **Successful deletion** — `code 0`.
- **Cannot delete active SSID** — `code 16`, unique to `delete_wifi_profile`. The reader refuses to delete the profile it is connected through.
- **SSID not found** — `code 15`, shared with `set_wifi`. The `ssid` is not a saved profile.

Notes and Caveats

DELETION IS PERMANENT

There is no undo. The saved profile and its stored credentials are removed; restoring it means re-provisioning with `set_wifi` and re-entering the credentials. Double-check the `ssid` before sending — the match is **case-sensitive** and exact.

- **The active profile is firmware-protected.** Unlike `set_wifi` with `connect: true` — which can switch networks and sever the control session — `delete_wifi_profile` **cannot** strand the reader: if you target the currently-connected `ssid`, the device refuses with `code 16` ("Cannot delete active SSID") rather than dropping its own connection. To retire the network the reader manages itself over, first move it to another profile (`set_wifi` `connect: true`), then delete the now-inactive profile.

Because the response is status-only, **verify the result with** `get_wifi` — confirm the deleted `ssid` no longer appears in the saved profiles and the connected profile is unchanged. For the concept-level view, see [Network Configuration](#).

Related:  [Network Configuration](#) ·  `get_wifi` ·  `set_wifi` ·  `delete_certificate`
 ·  [Response Codes](#)

